

Ejercicio: Sistema de Gestión de Empleados para una Empresa.

Contexto:

Considerando que eres un desarrollador de software independiente, la empresa "EcoTech Solutions" ha solicitado sus servicios para el desarrollo de una solución informática segura a sus requerimientos. EcoTech Solutions es una empresa en crecimiento que se especializa en tecnologías sostenibles. Han experimentado un rápido aumento en su fuerza laboral y necesitan una solución segura y efectiva para gestionar a sus empleados y proyectos. Es por esta razón que se han acercado a usted para desarrollar un nuevo Sistema de Gestión de Empleados con un enfoque en programación orientada a objetos (POO), sin dejar de lado los temas asociados a la seguridad informática.

Requisitos del Sistema:

Registro de Empleados:

El sistema debe permitir a los administradores de recursos humanos registrar nuevos empleados. Cada empleado tendrá información personal como nombre, dirección, número de teléfono, dirección de correo electrónico, fecha de inicio de contrato y salario. Además, se debe asignar automáticamente un ID único a cada empleado.

- Gestión de Departamentos:

La empresa tiene varios departamentos, como Desarrollo Sostenible, Investigación y Desarrollo, Ventas, Recursos Humanos, etc. El sistema debe permitir la creación, edición, búsquedas y eliminación de departamentos. Cada departamento tendrá un nombre y un gerente asociado.

- Asignación de Empleados a Departamentos:

Los empleados pueden ser asignados a un departamento en particular. Cada empleado solo puede pertenecer a un departamento a la vez. El sistema debe permitir la asignación y reasignación de empleados a departamentos.

- Registro de Tiempo:

El sistema debe permitir a los empleados registrar las horas trabajadas. Los empleados deben poder ingresar la fecha, la cantidad de horas trabajadas y una breve descripción de las tareas realizadas. Estos registros de tiempo deben estar asociados a un empleado y a un proyecto específico.

- Gestión de Proyectos:

La empresa trabaja en varios proyectos a la vez. El sistema debe permitir la creación, edición y eliminación de proyectos. Cada proyecto tendrá un nombre, una descripción y una fecha de inicio.

- Asignación de Empleados a Proyectos:

Los empleados pueden ser asignados a uno o varios proyectos. El sistema debe permitir la asignación y desasignación de empleados a proyectos.

- Generación de Informes:

Los administradores deben poder generar informes que muestren la información de los empleados, proyectos, departamentos y registros de tiempo. Estos informes deben ser exportables en diferentes formatos, como PDF o Excel.

- Autenticación y Autorización Seguras:

Implemente un sistema de autenticación robusto con contraseñas seguras y que asegure que los usuarios solo tengan acceso a los módulos del sistema para los que están autorizados.

- Seguridad de Datos Sensibles:

Almacena datos personales de empleados de forma segura utilizando técnicas de cifrado adecuadas y sigue las regulaciones de privacidad de datos aplicables.

- Validación de Entradas Seguras:

Implementa una rigurosa validación de todas las entradas del usuario para prevenir ataques comunes.

Consideraciones Técnicas:

- Utiliza el paradigma de programación orientada a objetos para diseñar y desarrollar el sistema.
- Implementa clases para representar empleados, departamentos, proyectos y registros de tiempo.
- Utiliza herencia y polimorfismo de manera efectiva para evitar duplicación de código.
- Utiliza una base de datos para almacenar la información del sistema.
- Desarrolla una interfaz de usuario para que los usuarios puedan interactuar con el sistema de manera intuitiva.

Construcción del Diagrama de clases paso a paso

Vamos a desglosar la construcción del diagrama de clases de forma secuencial, explicando cada clase y las relaciones entre ellas. Esto te servirá como base para explicar cómo se construye un diagrama de clases a tus estudiantes o colegas.

1. Clase Empleado

- **Atributos:** nombre, direccion, telefono, email, fechaInicioContrato, salario, idEmpleado.
- **Métodos:** registrarTiempo().
- **Descripción:** Esta clase representa a un empleado en la empresa. Cada empleado tiene información personal y un ID único. El método registrarTiempo() permite al empleado registrar las horas trabajadas.

2. Clase Departamento

- **Atributos:** nombre, gerente.
- **Métodos:** agregarEmpleado(Empleado), eliminarEmpleado(Empleado), buscarEmpleado(int idEmpleado).
- **Relación con Empleado:** Un departamento tiene múltiples empleados. La relación es una agregación, ya que los empleados son parte del departamento pero pueden existir independientemente. El Departamento contiene una referencia al gerente, que es de tipo Empleado.

3. Clase Proyecto

- **Atributos:** nombre, descripcion, fechaInicio.
- **Métodos:** agregarEmpleado(Empleado), eliminarEmpleado(Empleado).
- **Relación con Empleado:** Un proyecto puede tener múltiples empleados asignados. La relación es de agregación, ya que los empleados pueden estar asignados a varios proyectos.

4. Clase RegistroTiempo

- **Atributos:** fecha, horasTrabajadas, descripcion, empleado, proyecto.
- **Descripción:** Representa el registro de horas trabajadas por un empleado en un proyecto específico.
- **Relación con Empleado:** Un registro de tiempo está asociado a un empleado, creando una relación de dependencia (composición) porque el registro no tiene sentido sin el empleado.

- **Relación con Proyecto:** Un registro de tiempo está también relacionado con un proyecto, mostrando el trabajo hecho por el empleado en dicho proyecto.

5. Clase Usuario

- **Atributos:** username, password.
- **Métodos:** iniciarSesion(), cerrarSesion().
- **Descripción:** Representa a cualquier usuario del sistema, proporcionando atributos básicos para la autenticación.

6. Clase Administrador (Hereda de Usuario)

- **Métodos:** generarInforme().
- **Descripción:** Esta clase es una especialización de Usuario. Representa a los administradores que tienen permisos adicionales para gestionar empleados, departamentos, proyectos y generar informes.

7. Clase Informe

- **Métodos:** generarInformeEmpleado(Empleado), generarInformeProyecto(Proyecto), generarInformeDepartamento(Departamento), generarInformeTiempo(RegistroTiempo).
- **Relación con Administrador:** La clase Informe se asocia con Administrador, ya que los administradores generan informes basados en empleados, proyectos, departamentos y registros de tiempo.

8. Clase SistemaAutenticacion

- **Métodos:** autenticarUsuario(String username, String password), autorizarUsuario(Usuario, String permiso).
- **Relación con Usuario:** Usuario interactúa con SistemaAutenticacion para autenticarse y obtener autorización para acceder a diferentes módulos del sistema.

9. Clase SistemaEncriptacion

- **Métodos:** cifrarDatos(String datos), descifrarDatos(String datosCifrados).
- **Relación con Administrador:** La clase Administrador se asocia con SistemaEncriptacion para manejar el cifrado y la seguridad de los datos sensibles.
- **Relación con SistemaAutenticacion:** SistemaAutenticacion también interactúa con SistemaEncriptacion para asegurar las credenciales y otra información sensible.

10. Clase ValidacionEntrada

- **Métodos:** validarTexto(String texto), validarNumero(String numero), validarFecha(Date fecha).
- **Relación con Empleado:** La clase Empleado utiliza ValidacionEntrada para asegurar que los datos ingresados sean seguros y correctos. Este componente es crítico para prevenir ataques comunes como inyecciones de código.

Resumen de las Relaciones:

1. Agregación:

- Departamento → Empleado: Un departamento agrupa empleados.
- Proyecto → Empleado: Un proyecto agrupa empleados.

2. Composición:

- RegistroTiempo → Empleado: Un registro de tiempo depende de un empleado.
- RegistroTiempo → Proyecto: Un registro de tiempo depende de un proyecto.

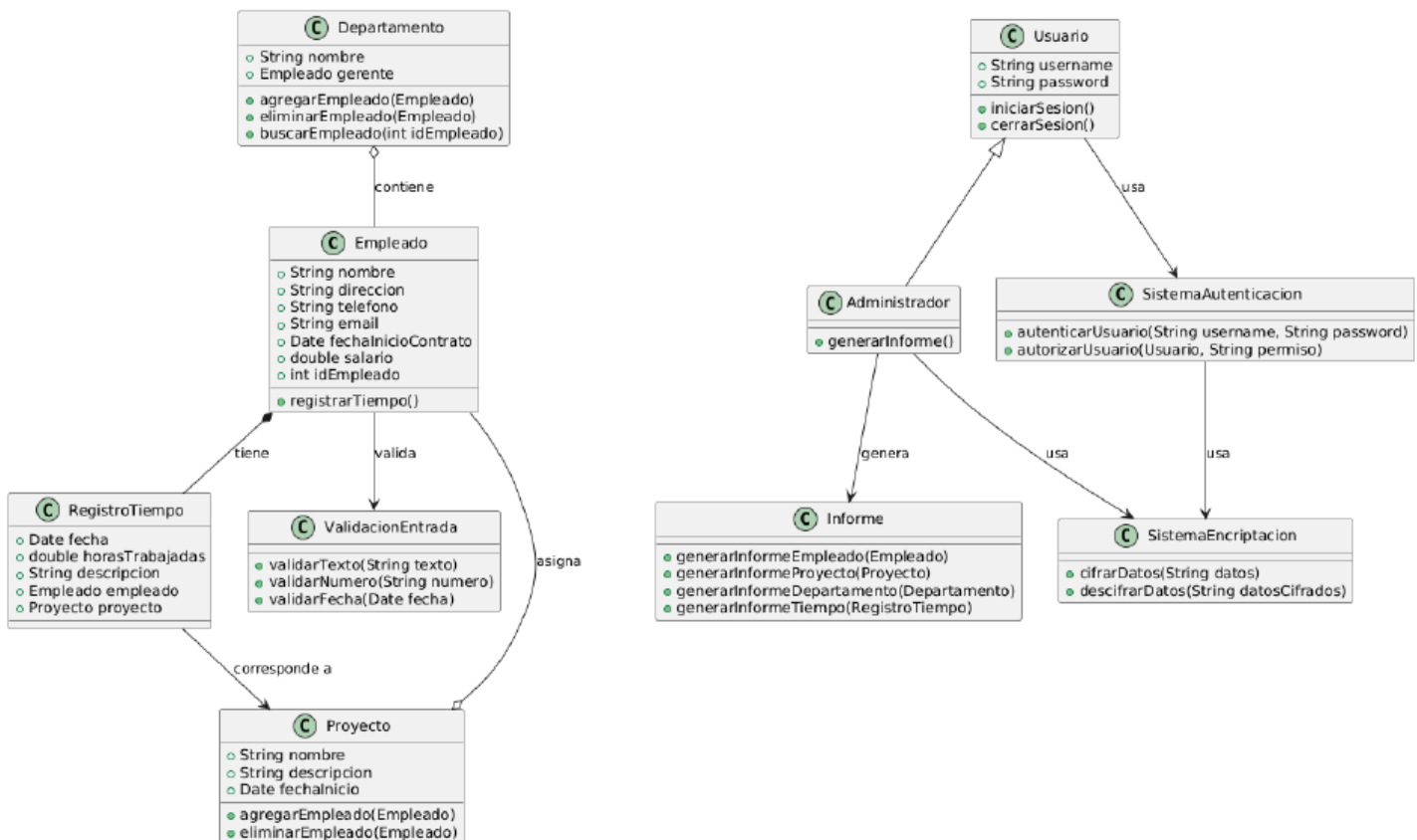
3. Herencia:

- Administrador hereda de Usuario.

4. Asociaciones:

- Administrador → Informe: Los administradores generan informes.
- Usuario → SistemaAutenticacion: Los usuarios se autentican a través del sistema.
- SistemaAutenticacion → SistemaEncriptacion: El sistema de autenticación usa encriptación.
- Empleado → ValidacionEntrada: Los empleados validan datos.

A continuación, el diagrama de clases completo:



Desarrollando Cardinalidad

Vamos a desglosar la cardinalidad de las relaciones entre las clases del diagrama. La cardinalidad indica cuántas instancias de una clase pueden estar asociadas con una instancia de otra clase. A continuación, describo la cardinalidad para cada relación entre clases:

1. Relación Departamento ↔ Empleado

- **Cardinalidad:**
 - Del lado de Departamento: 1 (un departamento puede tener **muchos** empleados).
 - Del lado de Empleado: 0..1 (un empleado pertenece a **un solo** departamento o puede no pertenecer a ninguno).
- **Representación:** 1 Departamento o-- * Empleado.

2. Relación Proyecto ↔ Empleado

- **Cardinalidad:**
 - Del lado de Proyecto: 1 (un proyecto puede tener **muchos** empleados asignados).
 - Del lado de Empleado: * (un empleado puede estar asignado a **varios** proyectos simultáneamente).
- **Representación:** 1 Proyecto o-- * Empleado.

3. Relación Empleado ↔ RegistroTiempo

- **Cardinalidad:**
 - Del lado de Empleado: 1 (un empleado puede tener **muchos** registros de tiempo).
 - Del lado de RegistroTiempo: 1 (un registro de tiempo pertenece a **un solo** empleado).
- **Representación:** 1 Empleado *-- * RegistroTiempo.

4. Relación RegistroTiempo ↔ Proyecto

- **Cardinalidad:**
 - Del lado de RegistroTiempo: 1 (un registro de tiempo se asocia con **un solo** proyecto).
 - Del lado de Proyecto: 1 (un proyecto puede tener **muchos** registros de tiempo asociados).
- **Representación:** 1 RegistroTiempo --> 1 Proyecto.

5. Relación Usuario ↔ SistemaAutenticacion

- **Cardinalidad:**
 - Del lado de Usuario: * (muchos usuarios pueden utilizar el sistema de autenticación).
 - Del lado de SistemaAutenticacion: 1 (un sistema de autenticación se comparte entre muchos usuarios).
- **Representación:** * Usuario --> 1 SistemaAutenticacion.

6. Relación Administrador ↔ Informe

- **Cardinalidad:**
 - Del lado de Administrador: 1 (un administrador puede generar **muchos** informes).
 - Del lado de Informe: 0..1 (un informe es generado por **un solo** administrador).
- **Representación:** 1 Administrador --> * Informe.

7. Relación SistemaAutenticacion ↔ SistemaEncriptacion

- **Cardinalidad:**
 - Del lado de SistemaAutenticacion: 1 (el sistema de autenticación puede utilizar **un solo** sistema de encriptación).
 - Del lado de SistemaEncriptacion: 1 (un sistema de encriptación puede ser utilizado por **muchos** sistemas de autenticación).

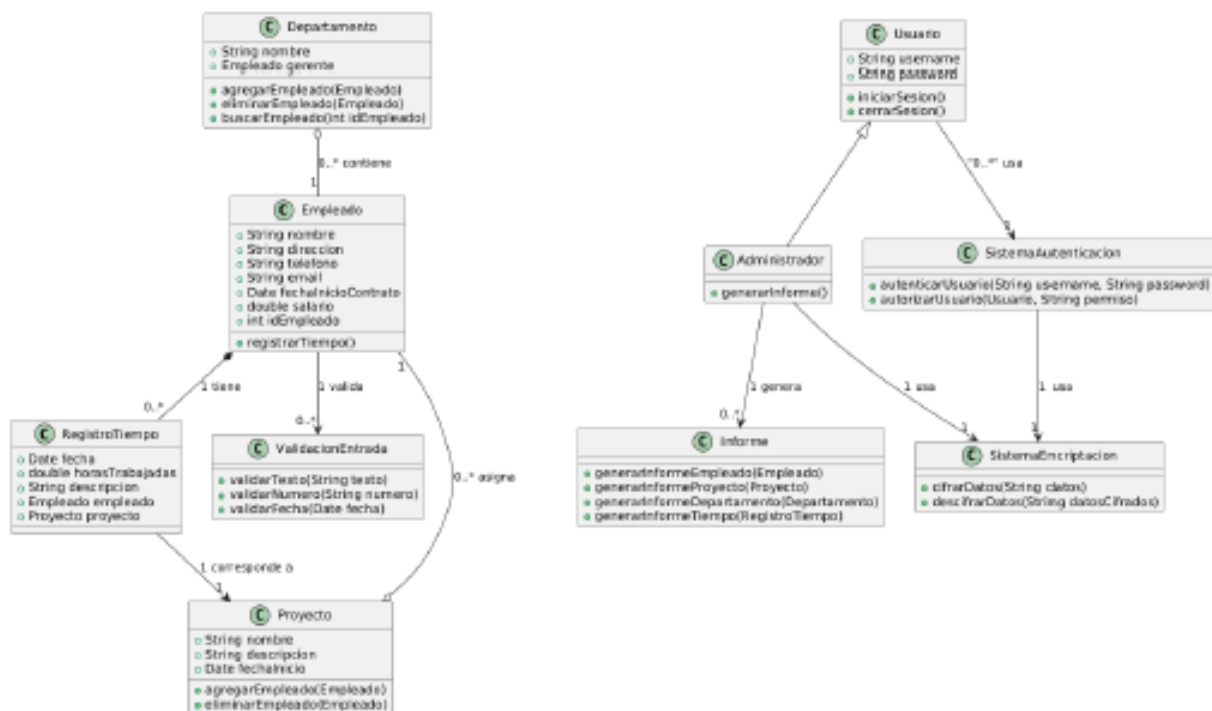
- **Representación:** 1 SistemaAutenticacion → 1 SistemaEncriptacion.

8. Relación Empleado ↔ ValidacionEntrada

- **Cardinalidad:**
 - Del lado de Empleado: * (muchos empleados pueden utilizar el sistema de validación).
 - Del lado de ValidacionEntrada: 1 (un sistema de validación puede ser utilizado por muchos empleados).
- **Representación:** * Empleado → 1 ValidacionEntrada.

9. Relación de Herencia Usuario ↔ Administrador

- **Cardinalidad:**
 - Del lado de Usuario: 1 (cada Administrador es un tipo de Usuario).
 - Del lado de Administrador: 0..1 (un administrador es un tipo específico de usuario).
- **Representación:** Usuario <|-- Administrador.



Detalles de la Cardinalidad:

1. **Departamento o-- "1" Empleado : "0.. contiene"**
 - Un Departamento contiene 0 o más Empleado(s), pero cada Empleado pertenece a exactamente un Departamento.
2. **Proyecto o-- "1" Empleado : "0.. asigna"**
 - Un Proyecto puede asignar 0 o más Empleado(s), y cada Empleado puede estar asignado a varios proyectos.
3. **Empleado -- "0.." RegistroTiempo : "1 tiene"**
 - Un Empleado puede tener 0 o más RegistroTiempo(s), y cada RegistroTiempo pertenece a un solo Empleado.
4. **RegistroTiempo → "1" Proyecto : "1 corresponde a"**

- Un RegistroTiempo corresponde a un solo Proyecto, y un Proyecto puede tener varios RegistroTiempo(s).
- 5. **Administrador** → "0..*" **Informe** : "1 genera"
 - Un Administrador puede generar 0 o más Informe(s), pero cada Informe es generado por un solo Administrador.
- 6. **Usuario** → "1" **SistemaAutenticacion** : "0..*" **usa**
 - Muchos Usuario(s) pueden utilizar el SistemaAutenticacion, pero el SistemaAutenticacion es único para todos.
- 7. **SistemaAutenticacion** → "1" **SistemaEncriptacion** : "1 usa"
 - El SistemaAutenticacion usa un SistemaEncriptacion, y el SistemaEncriptacion es único para todos.
- 8. **Empleado** → "0..*" **ValidacionEntrada** : "1 valida"
 - Muchos Empleado(s) pueden usar la ValidacionEntrada, que es única para todos.

Aclaración

La relación entre Administrador y Usuario es de **herencia**, ya que Administrador es una especialización de Usuario. En este tipo de relación, no se suele expresar la cardinalidad de la misma manera que en las asociaciones normales, ya que la cardinalidad inherente de la herencia es "uno a uno" para cada instancia de la subclase con respecto a la superclase.

Cardinalidad en la relación de herencia:

- **Del lado de Usuario:** 1 (cada Administrador es un Usuario).
- **Del lado de Administrador:** 0..1 (un Usuario puede ser o no un Administrador específico).

Interpretación:

- Si consideramos que Administrador hereda de Usuario, la relación implícita es que cualquier instancia de Administrador también es una instancia de Usuario. Por lo tanto, cada Administrador cuenta como un Usuario, pero no todos los Usuario(s) son necesariamente Administrador(es).

En resumen:

La cardinalidad en la relación de herencia entre Administrador y Usuario podría considerarse como:

- **Usuario:** 1 (cada Administrador es un Usuario).
- **Administrador:** 0..1 (un Usuario puede ser o no un Administrador).

Sin embargo, en diagramas UML, esta cardinalidad no suele especificarse explícitamente en relaciones de herencia, ya que se entiende implícitamente que una instancia de la subclase es siempre una instancia de la superclase.